Introduction to Algorithms

What is an Algorithm? - An algorithm is a procedure to accomplish a specific task. It takes any of the possible input instances and transforms it to the desired output through a sequence of computational steps.

Let us consider the problem of preparing an omelet. To prepare, we follow these steps:

1) Get the frying pan.
2) Get the oil.
    a. Do we have oil?
        i. If yes, put it in the pan.
        ii. If no, do we want to buy oil?
            1. If yes, then go out and buy.
            2. If no, we can terminate.
3) Turn on the stove, etc...

There are two main criteria for judging the merits of algorithms:

- Correctness (does the algorithm solve the problem in a finite number of steps?)

An algorithm for a computational problem is correct if every problem instance provided as input halts or finishes its computing in finite time and outputs the correct solution to the problem instance. A problem must be defined precisely before you can prove an algorithm correct. That means, it must be specified 1) what inputs are allowed, and 2) what the output must satisfy. You can prove an algorithm is incorrect by finding good counter-examples.

- Efficiency (how much resources in terms of memory and time does it take to execute?)

To judge efficiency, we look at how these resource requirements grow as the input size increases, rather than exact running times on a specific computer. An algorithm may be considered inefficient if another algorithm can solve the same problem using fewer resources, especially for large inputs.

Formal Algorithm Definition: A finite set of statements that guarantees an optimal solution in a finite interval of time

Examples of Real-Life Algorithms:

i) The internet allows people all over the world to easily and quickly access a lot of information. Smart algorithms help websites manage this huge amount of data by finding the best paths for the data to travel. They also use search engines to quickly find pages that contain specific information.

ii) You need to compress a large file containing text to occupy less space. Algorithms such as Huffman coding allow you to store with fewer bits.

Algorithm Specification:

- Input - Every algorithm must take zero or more number of input values from the external.
- Output - Every algorithm must produce an output as a result.
- Definiteness - Every statement/instruction in an algorithm must be unambiguous (only one interpretation)
- Finiteness - The algorithm must produce results within a finite number of steps for all cases.
- Effectiveness - Every instruction must be basic enough to be carried out, and it must also be feasible.

To analyze algorithms, we need to predict the amount of resources required:

- Memory: how much space is needed?
- Computational time: how fast the algorithm runs

If computers were infinitely fast and had infinite memory, any correct method for solving a problem would do. You would probably want your implementation to be within the bounds of good software engineering practice (for example, your implementation should be well-designed and documented), but you would most often use whichever method was the easiest to implement. description should allow a competent but skeptical programmer to quickly and correctly implement the algorithm in their favorite programming language, without understanding why it works.

Describing the Algorithm:

Algorithms are abstract, precise procedures, not programs themselves. Programs are just concrete implementations of algorithms in a specific language.

Because algorithms must be clear and unambiguous, plain English alone is too vague, while real code is too detailed and rigid for explanation. The best way to describe an algorithm is pseudocode, which combines the structure of programming and mathematics with clear, simple language to express each step precisely.

```
Algorithm FindMaximum

    Input: list of numbers L
    Output: largest number in L

    max ← L[0]

    for each number x in L do
        if x > max then
            max ← x
        end if
    end for

    return max

End Algorithm
```

Additional Notes:

- Even if you never have to explain your algorithms to anyone else, it's still important to develop them with an audience in mind. Your primary job as an algorithm designer is teaching other people how and why your algorithms work.

- Algorithmic problems are often presented using standard English, in terms of real-world objects. It's up to us, the algorithm designers, to restate these problems in terms of formal, abstract, mathematical objects such as numbers, arrays, lists, graphs, trees, and so on that we can reason about formally.